

Apache Beam reduces the complexity of processing massive amounts of data. It is widely used by thousands of businesses across the world thanks to its innovative data processing tools, proven scalability, and robust but scalable features.

The key features of Apache Beam are:

- Powerful abstraction
- Unified batch and streaming programming model
- Cross-language capabilities
- Portability
- Extensibility
- Flexibility
- Ease of adoption

Apache Beam is also available in its cloud based variant called Apache Beam Playground (<https://play.beam.apache.org/>), with the help of which programs can be executed online (Figure 4) using a range of programming languages including Java, Python, Go and SCIO.

Installation of Apache Beam on a dedicated server

Apache Beam can be installed on a dedicated server or Google Colab, as follows:

```
! pip install apache-beam
```

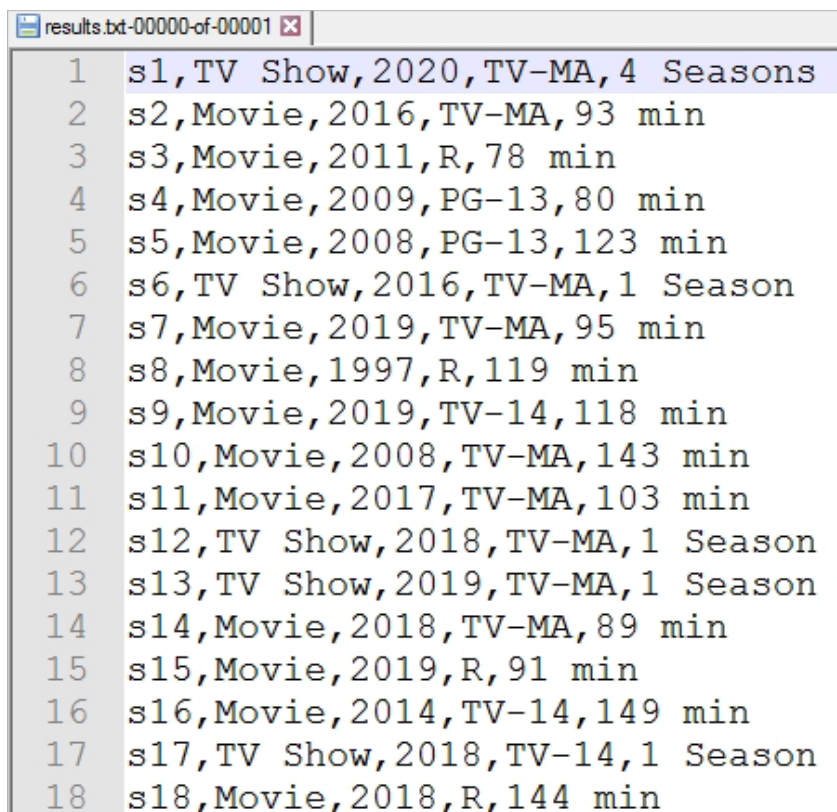
To work with data pipelines, it uses the following segments.

- *Pipeline*: Reading, Processing, Storage of data
- *PCollection*: Data frames, Accessing data sets
- *PTransform*: Data transformation, Windowing, Watermarks, Time stamps
- *Runner*: Operation of the pipelines

Here is a scenario where a data pipeline is executed on a CSV movies data set that has the following attributes:

1. movie-id
2. type-of-show (movie/web series)
3. release-year
4. duration-of-the-movie

The following code can be



```

1 s1,TV Show,2020,TV-MA,4 Seasons
2 s2,Movie,2016,TV-MA,93 min
3 s3,Movie,2011,R,78 min
4 s4,Movie,2009,PG-13,80 min
5 s5,Movie,2008,PG-13,123 min
6 s6,TV Show,2016,TV-MA,1 Season
7 s7,Movie,2019,TV-MA,95 min
8 s8,Movie,1997,R,119 min
9 s9,Movie,2019,TV-14,118 min
10 s10,Movie,2008,TV-MA,143 min
11 s11,Movie,2017,TV-MA,103 min
12 s12,TV Show,2018,TV-MA,1 Season
13 s13,TV Show,2019,TV-MA,1 Season
14 s14,Movie,2018,TV-MA,89 min
15 s15,Movie,2019,R,91 min
16 s16,Movie,2014,TV-14,149 min
17 s17,TV Show,2018,TV-14,1 Season
18 s18,Movie,2018,R,144 min
  
```

Figure 5: Filtered results from Apache Beam

applied to filter and fetch the records matching 'Movie':

```

import apache_beam as apachebeam
datapipeline = apachebeam.Pipeline()
moviesdataset = (
    datapipeline
    | apachebeam.
    io.ReadFromText("moviesdataset.csv",
    skip_header_lines=1)
    | apachebeam.Map(lambda line:line.
    split(","))
    | apachebeam.Filter(lambda
    line:line[1] == "Movie")
    | apachebeam.
    io.WriteToText("results2.txt")
    )
datapipeline.run()
  
```

Pipeline operations on a large data set can be implemented with minimum lines of code using Apache Beam.

Modern data engineering scenarios make use of Snowflake platforms that have enhanced structured formats for implementing data pipelines. Snowflake's data pipelines are flexible, allowing for either batch or continuous processing. Data transformation and optimisation for continuous data loading are typically labour-intensive tasks; however, modern data pipelines automate many of these processes **END** 🐧

 By: Dr Gaurav Kumar

The author is associated with various academic and research institutes for delivering expert lectures and conducting technical workshops on the latest technologies and tools.